Lesson 2.4 **Duration: 40 minutes**

Dictionaries

Learning Objectives

- Define dictionaries as key-value pairs and explain their advantages over lists
- Create dictionaries, access values by key, and add or remove entries
- Choose appropriate data structures for different problem scenarios

Engage (5-7 minutes)

A phone book maps names to phone numbers—each name uniquely identifies a person. Dictionaries work the same way in programming: they store data as key-value pairs where each key maps to exactly one value. Unlike lists that use numeric indices, dictionaries use meaningful keys. Today we learn this powerful data structure that underpins databases, configuration files, and JSON data.

Explore (10-12 minutes)

Create a dictionary representing a student with keys: name, age, grade, and subjects (a list). Access the name, then update the grade. Add a new key "address" and remove the "age" key. Now create a dictionary of five words with their meanings. Try to access a key that does not exist—what happens? How is this different from accessing an invalid list index?

Explain (10-12 minutes)

A dictionary is an unordered (in Python 3.7+, insertion-ordered) collection of key-value pairs. Keys must be immutable (strings, numbers, tuples) and unique, while values can be any data type. Common operations include: `dict[key]` to access, `dict[key] = value` to set, `del dict[key]` to remove, and `key in dict` to check existence. Dictionaries provide O(1) average time complexity for lookups, making them much faster than searching through lists. They are ideal for representing structured data and relationships.

Elaborate (8-10 minutes)

Build a dictionary-based phonebook that supports: adding contacts, searching by name, deleting contacts, listing all contacts alphabetically, and finding contacts by partial name match. Then create a word frequency counter that takes a sentence and returns a dictionary with words as keys and counts as values. Implement a dictionary that maps student IDs to their grades, with functions to add, update, and calculate class average. Discuss: when would you use a dictionary instead of a list of tuples?

Evaluate (5-8 minutes)

Given the dictionary `{"name": "Rahul", "age": 15, "grade": 9, "marks": [85, 90, 78]}`, write the output of: accessing `["name"]`, adding `"city": "Mumbai"`, removing `"age"`, using `.get("phone", "N/A")`, and iterating through keys. Explain why dictionary lookups are faster than list searches.